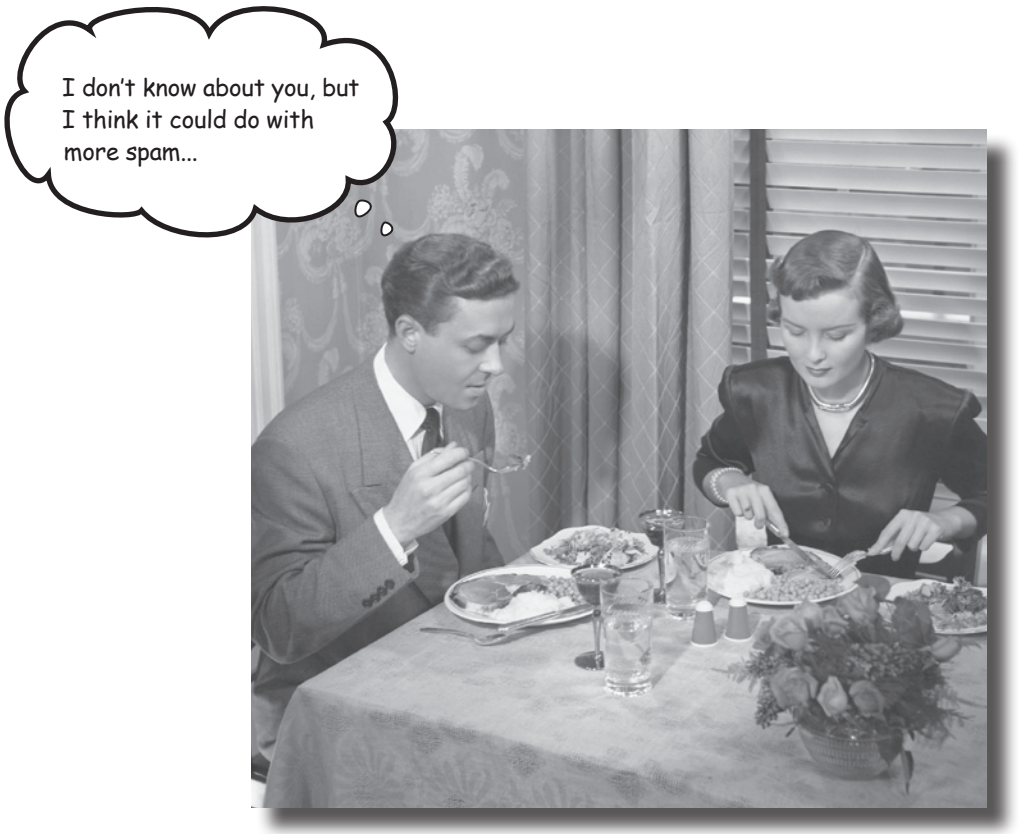
The Top Ten Things (we didn't cover)



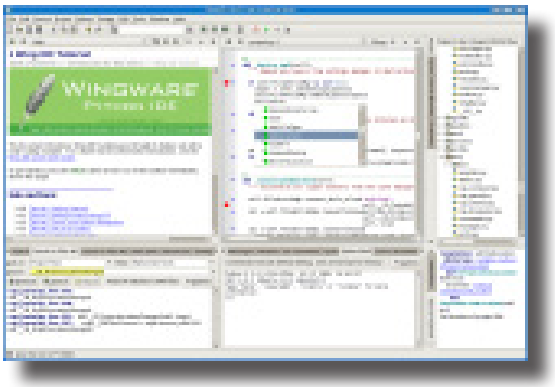
You've come a long way.

But learning about Python is an activity that never stops. The more Python you code, the more you'll need to learn new ways to do certain things. You'll need to master new tools and new techniques, too. There's just not enough room in this book to show you everything you might possibly need to know about Python. So, here's our list of the top ten things we didn't cover that you might want to learn more about next.

#1: Using a "professional" IDE

Throughout this book, you've used Python's **IDLE**, which is great to use when first learning about Python and, although it's a little quirky, can handle most programming tasks. It even comes with a built-in **debugger** (check out the Debug menu), which is surprisingly well equipped. Chances are, however, sooner or later, you'll probably need a more full-featured integrated development environment.

One such tool worth looking into is the **WingWare Python IDE**. This professional-level development tool is specifically geared toward the Python programmer, is written by and maintained by Python programmers, and is *itself* written in Python. **WingWare Python IDE** comes in various licencing flavor: it's free if you're a student or working on an open source project, but you'll need to pay for it if you are working within a for-profit development environment.



Written in Python by Python programmers for other Python programmers...what else could you ask for?

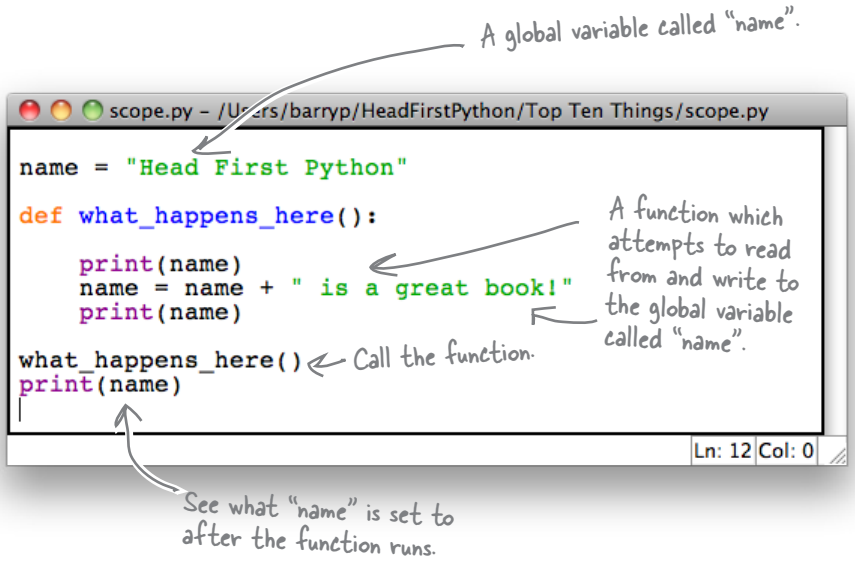
The WingWare Python IDE

More general tools also exist. If you are running Linux, the **KDevelop IDE** integrates well with Python.

And, of course,there are all those *programmer editors* which are often all you'll ever need. Many Mac OS X programmers swear by the **TextMate** programmer's editor. There's more than a few Python programmers using **emacs** and **vi** (or its more common variant, **vim**). Your author is a huge fan of **vim**, but also spends large portions of his day using **IDLE** and the Python shell.

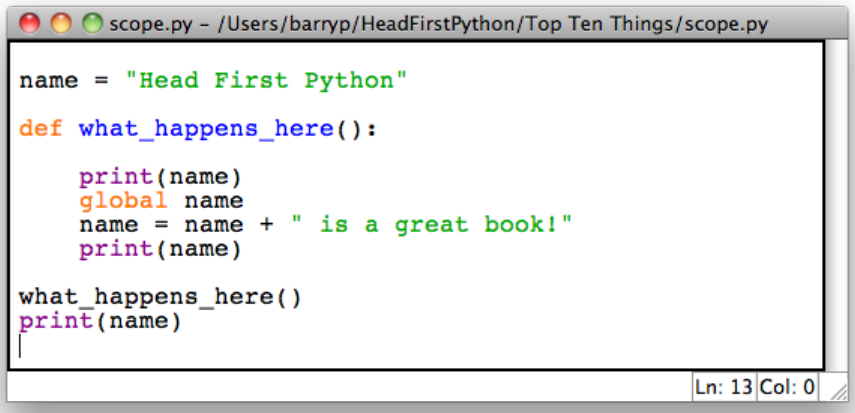
#2: Coping with scoping

Consider the following short program:



If you try to run this program, Python complains with this error message: *UnboundLocalError: local variable 'name' referenced before assignment*…whatever that means!

When it comes to scope, Python is quite happy to let you access and read the value of a global variable within a function, **but you cannot change it**. When Python sees the assignment, it looks for a local variable called `name`, doesn't find it, and throws a *hissy fit* and an `UnboundLocalError` exception. To access *and* change a global variable, you must explicitly declare that's your intention, as follows:



Some programmers find this quite ugly. Others think it's what comes to pass when you watch Monty Python reruns while designing your programming language. No matter what everyone thinks: this is what we're stuck with! ☺

#3: Testing

Writing code is one thing, but **testing** it is quite another. The combination of the Python shell and IDLE is great for testing and experimenting with small snippets of code, but for anything substantial, a testing framework is a must.

Python comes with *two* testing frameworks out of the box.

The first is familiar to programmers coming from another modern language, because it's based on the popular *xUnit* testing framework. Python's `unittest` module (which is part of the standard library) lets you create test code, test data, and a test suite for your modules. These exist in separate files from you code and allow you to exercise your code in various ways. If you already use a similar framework with your current language, rest assured that Python's implementation is essentially the same.

The other testing framework, called `doctest`, is also part of the standard library. This framework allows you to take the output from a Python shell or IDLE session and use it as a test. All you need to do is copy the content from the shell and add it to your modules *documentation strings*. If you add code like this to the end of your modules, they'll be ready for "doctesting":

```
if __name__ == "__main__":
    import doctest
    doctest.testmod()
```

If your code is imported as a module, this code does NOT run. If you run your module from the command line, your tests run.

If you then run your module at your operating systems comand line, your tests run. If all you want to do is import your module's code and not run your tests, the previous `if` statement supports doing just that.

For more on `unittest` and `doctest`, search the online Python documentation on the Web or via IDLE's *Help* menu.

#4: Advanced language features

With a book like this, we knew we'd never get to cover the entire Python language unless we tripled the page count.

And let's face it, no one would thank us for that!

There's a lot more to Python, and as your confidence grows, you can take the time to check out these advanced language features:

Anonymous functions: the `lambda` expression lets you create small, one-line, non-named functions that can be incredibly useful once you understand what's going on.

Generators: like iterators, generators let you process sequences of data. Unlike iterators, generators, through the use of the `yield` expression, let you minimize the amount of RAM your program consumes while providing iterator-like functionality on large datasets.

Custom exceptions: create your own exception object based on those provided as standard by Python.

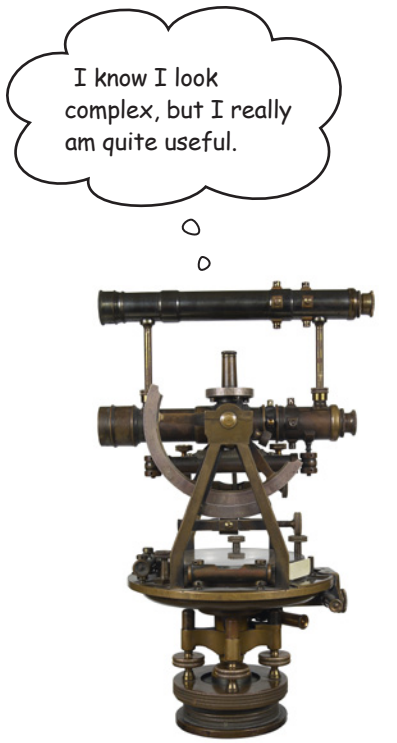
Function decorators: adjust the behavior of a preexisting function by hooking into its start-up and teardown mechanisms.

Metaclasses: custom classes that themselves can create custom classes. These are really only for the truely brave, although you did use a metaclass when you created your Sightings form using the *Django form validation framework* in Chapter 10.

Most (but not all) of these language features are primarily of interest to the Python programmer building tools or language extensions for use by other Python programmers.

You might never need to use some of these language features in your code, but they are all worth knowing about. Take the time to understand *when* and *where* to use them.

See **#10** of this appendix for a list of my favorite Python books (other than this one), which are all great starting points for learning more about these language features.



#5: Regular expressions

When it comes to working with textual data, Python is a bit of a natural. The built-in `string` type comes with so many methods that most of the standard string operations such as finding and splitting are covered. However, what if you need to extract a specific part of a string or what if you need to search and replace within a string based on a specific specification? It is possible to use the built-in string methods to implement solutions to these types of problems, but—more times than most people would probably like to admit to—using a **regular expression** works better.

Jeff Friedl's regular expression "bible", which is well worth a look if you want to learn more. Look up the "re" module in Python's does, too.

Consider this example, which requires you to extract the area code from the `phone_number` string and which uses the built-in string methods:

area_code.py – /Users/barryp/HeadFirstPython/Top Ten Things/area_code.py

```
phone_number = "Home: (555) 265-2901"

start = phone_number.find('(')

start = start+1
end = start+3

area_code = phone_number[start:end]

print('The area code is: ' + area_code)
```

Find the opening "(".

Calculate where the area code is in the string.

Extract the area code.

Ln: 12 Col: 0

This code works fine, but it *breaks* when presented with the following value for `phone_number`:

```
phone_number = "Cell (mobile): (555)-918-8271"
```

Why does this phone number cause the program to fail? Try it and see what happens...

When you use a **regular expression**, you can specify *exactly* what it is you are looking for and *improve the robustness* of your code:

area_code_re.py – /Users/barryp/HeadFirstPython/Top Ten Things/area_code_re.py

```
import re

phone_number = "Home: (555) 265-2901"

results = re.search('\((\d{3})\)', phone_number)
area_code = results.group(1)

print('The area code is: ' + area_code)
```

Ln: 10 Col: 0

This looks a little strange, but this regular expression is looking for an opening "(" followed by three digits and then a closing ")". This specification is much more likely to find the area code and won't break as quickly as the other version of this program.

#6: More on web frameworks

When it comes to building web applications, CGI works, but it's a little old-fashioned. As you saw in Chapter 10, Google's App Engine technology supports CGI, but also WSGI and a number of web framework technologies. If you aren't deploying to the cloud and prefer to roll your own, you have plenty of choices. What follows is a representative sample. My advice: try a few on for size and see which one works best for you.

Search for the following terms in your favorite search engine: **Django**, **Zope**, **TurboGears**, **Web2py**, and **Pylons**.

#7: Object relational mappers and NoSQL

Working with SQL-based databases in Python is well supported, with the inclusion of SQLite in the standard library a huge boon. Of course, the assumption is you are familiar with SQL and happy to use SQL to work with your data.

But what if you aren't? What if you *detest* SQL?

An **object relational mapper** (ORM) is a software technology that lets you use an underlying SQL-based database *without* having to know anything about SQL. Rather than the procedural interface based on the Python database API, ORMs provide an object-oriented interface to your data, exposing it via *method calls* and *attribute lookups* as opposed to columns and rows.

Many programmers find ORMs a much more natural mechanism for working with stored datasets and the Python community creates and supports a number of them.

One of the most interesting is **SQL Alchemy**, which is popular and included in a number of the web framework technologies discussed in **#6**. Despite being hugely popular anyway, **SQL Alchemy** is also interesting because it supports *both* Python 2 and Python 3, which makes it a standout technology (for now).

If you find yourself becoming increasingly frustrated by SQL, check out an ORM. Of course, you have already experienced a similar technology: *Google App Engine's* **datastore** API is very similar in style to those APIs provided by the major Python ORMs.

There's NoSQL, too.

In addition to database technologies that let you avoid working with the underlying SQL-based database, a new breed of technologies have emerged that let you *drop* your SQL database in its entirety. Known collectively as **NoSQL**, these data tools provide an alternative non-SQL API to your data and do not use an SQL-based database management system at all. As these technologies are relatively new, there's been more activity around Python 2 than Python 3, but they are still worth checking out. **CouchDB** and **MongoDB** are the two most closely associated with robust Python implementations. If you like working with your data in a Python dictionary and wished your database technology let you store your data in much the same way, then you need to take a look at **NoSQL**: *it's a perfect fit*.

#8: Programming GUIs

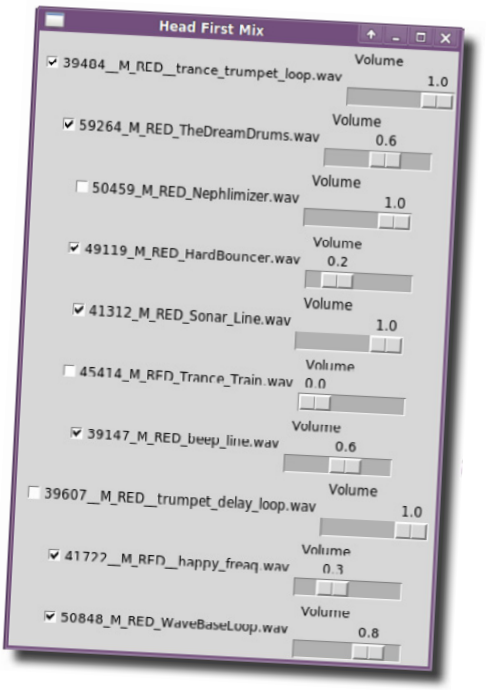
In this book, you've created text-based interfaces, web-based interfaces and interfaces that ran on Android devices. But what if you want to create a desktop application that runs on your or your user's desktop computer? Are you out of luck, or can Python help here, too?

Well…as luck would have it, Python comes preinstalled with a GUI-building toolkit called **tkinter** (shorthand for *Tk Interface*). It's possible to create a usable and useful graphical user interface (GUI) with **tkinter** and deploy it on Mac OS X, Windows, and Linux. With the latest version of **Tk**, your developed app takes on the characteristics of the underlying operating system, so when you run on Windows, your app looks like a Windows desktop app, when it run on Linux, it looks like a Linux desktop app, and so on.

You write your Python and **tkinter** code *once*, then run it anywhere and it just works. There are lots of great resources for learning to program with **tkinter**, with one of the best being the last few chapters of *Head First Programming*, but since plugging *that* book would be totally shameless, I won't mention it again.

Other GUI-building technologies do exist, with the **PyGTK**, **PyKDE**, **wxPython**, and **PyQT** toolkits coming up in conversation more than most. Be warned, however, that most of these toolkits target Python 2, although support for Python 3 is on its way. Search the Web for any of the project names to learn more.

Oh, look: it's one of the GUIs created in "Head First Programming"...and yes, I said I wouldn't mention THAT book again, but isn't this GUI beautiful? ☺



#9: Stuff to avoid

When it comes to stuff to avoid when using Python, there's a very short list. A recent tweet on *Twitter* went something like this:

Threads do indeed exist in Python but should be **avoided** where possible.

This has *nothing* to do with the quality of Python's threading library and *everything* to do with Python's implementation, especially the implementation known as **CPython** (which is more than likely the one you're running now). Python is implemented using a technology known as the **Global Interpreter Lock** (**GIL**), which enforces a restriction that Python can only ever run on a single interpreter process, even in the presence of multiple processors.

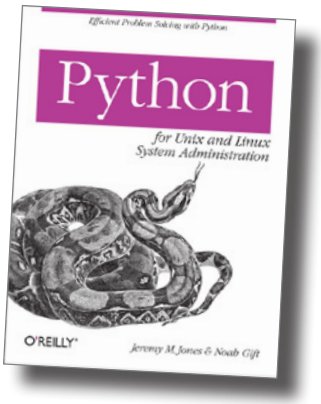
What all this means to you is that your beautifully designed and implemented program that uses threads will never run faster on multiple processors even if they exist, because *it can't use them*. Your threaded application will run **serially** and, in many cases, run *considerably slower* than if you had developed the same functionality without resorting to threads.

Main message: **don't use threads** with Python until the GIL restriction is removed…*if it ever is*.

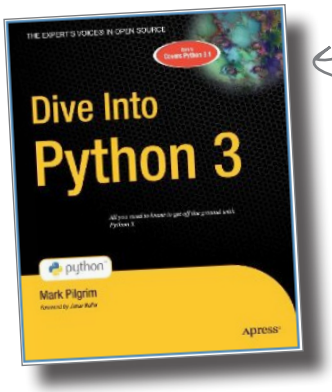
#10: Other books

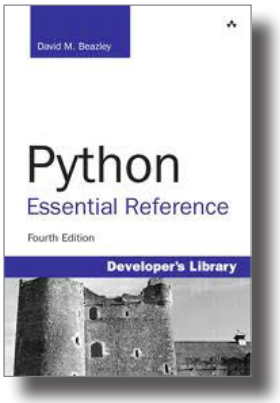
There are lots of great books that cover Python in general, as well as specifically within a particular problem domain. Here is a collection of my favorite Python books, which we have no hestitation in recommending to you.

If you are a sysadmin, then this is the Python book for you.

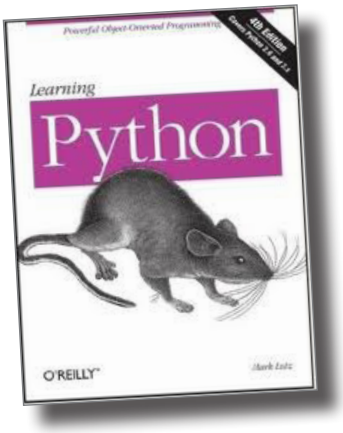


Includes a great case study involving the porting of a complex Python 2 module to Python 3.

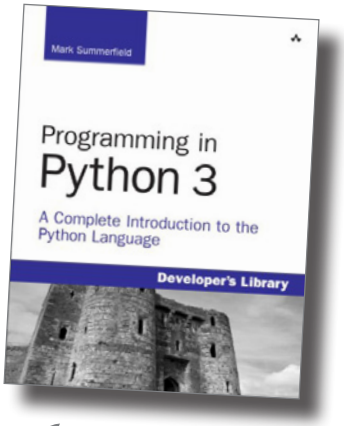




The best desktop reference on the market



At 1,200 pages, this is the definitive language reference for Python: it's got everything in it!



Includes some big examples with big technology: XML, parsing and advanced language features.

Index

Symbols and Numbers

A

B

C

D

E

F

G

H

I

J

K

L

M

N

O

P

Q

R

S

T

U

V

W

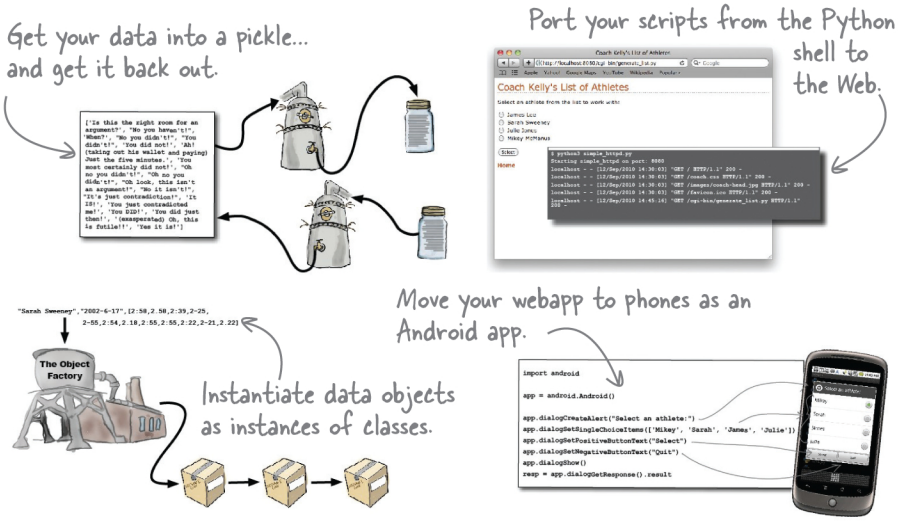
Y

Head First Python

Python/Programming Languages

What will you learn from this book?

Ever wished you could learn Python from a book? *Head First Python* helps you learn the language through a unique method that goes beyond syntax and how-to manuals. You'll quickly grasp Python's fundamentals, then move on to persistence, exception handling, web development, SQLite, data wrangling, and Google App Engine. You'll also learn how to write mobile apps for Android, all thanks to the power that Python gives you. *Head First Python* is a complete learning experience that will help you become a bona fide Python programmer.



Why does this book look so different?

We think your time is too valuable to waste struggling with new concepts. Using the latest research in cognitive science and learning theory to craft a multi-sensory learning experience, *Head First Python* uses a visually rich format designed for the way your brain works, not a text-heavy approach that puts you to sleep.

"*Head First Python* is a great introduction to not just the Python language, but Python as it's used in the real world. The book goes beyond the syntax to teach you how to create applications for Android phones, Google's App Engine, and more."

— David Griffiths, author and Agile coach

"Where other books start with theory and progress to examples, *Head First Python* jumps right in with code and explains the theory as you read along. The breadth of examples and explanation cover the majority of what you'll use in your job every day."

— Jeremy Jones, coauthor of Python for Unix and Linux System Administrators

US $49.99 CAN $57.99

ISBN: 978-1-449-38267-4

O'REILLY®

oreilly.com

headfirstlabs.com